

StringeeX Django — Senior Developer Reference

Project: Live Call Intelligence Dashboard for TradeX

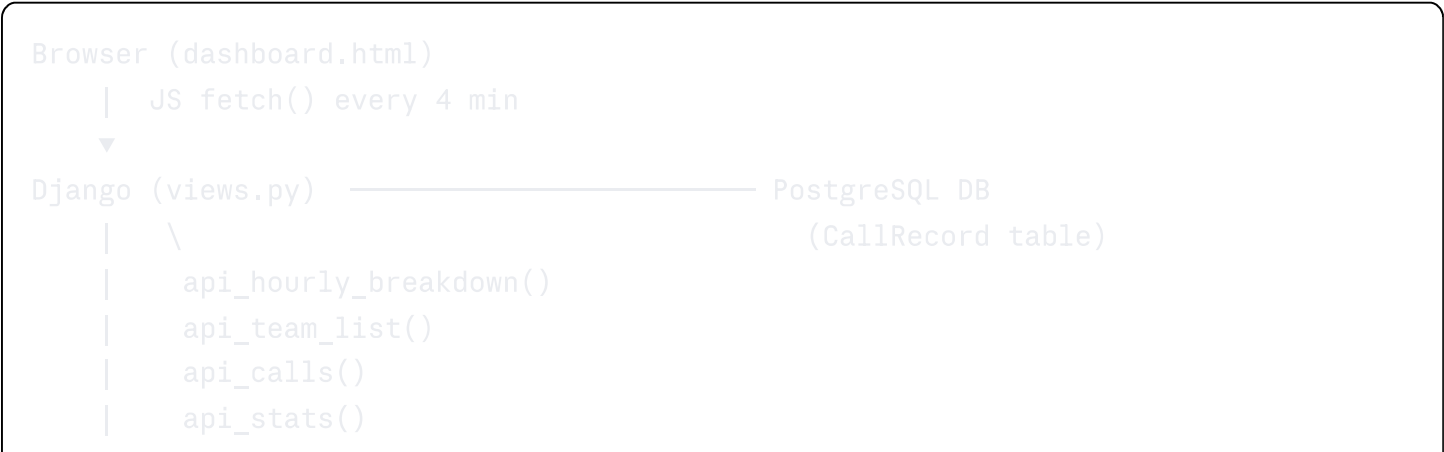
Stack: Django 4.2 · PostgreSQL · vanilla JS dashboard

Last updated: May 2026

Table of Contents

- 1. [Architecture Overview](#)
 - 2. [File Map & Responsibilities](#)
 - 3. [Environment & Configuration](#)
 - 4. [Database Model](#)
 - 5. [Team Lookup System](#)
 - 6. [Data Ingestion — API & CSV](#)
 - 7. [API Endpoints Reference](#)
 - 8. [Dashboard Frontend](#)
 - 9. [Access Control](#)
 - 10. [Timezone Handling](#)
 - 11. [Debugging Runbook](#)
 - 12. [Team List Updates](#)
 - 13. [Adding New Panels & Endpoints](#)
 - 14. [Data Change Operations](#)
 - 15. [Deployment & Process Management](#)
 - 16. [Common Gotchas & Known Quirks](#)
-

1. Architecture Overview



```

|
├─ team_lookup.py    ← data/Team_tradex.xlsx (in-memory cache)
|
├─ Management Commands
    sync_calls --daemon    ← StringeeX API (bluewave2, bluewave3)
    sync_calls --csv       ← Manual CSV export
    reenrich_teams        ← Re-apply Excel changes to existing rows

```

Data flow:

1. `sync_calls --daemon` runs as a background process, polling the StringeeX API every 4 minutes.
2. Each call record is enriched with team metadata (TL, Pool, Full Name) from the Excel at write time.
3. The dashboard JS hits `/api/hourly-breakdown/` every 4 minutes and re-renders the table for today.
4. The team sidebar hits `/api/team-list/` once on page load — it merges Excel data with DB data.

2. File Map & Responsibilities

```

stringeex_django/
├─ .env                ← All secrets and config (never commit)
├─ requirements.txt    ← django, psycopg2-binary, python-dotenv,
requests
├─ manage.py
|
├─ data/
|   └─ Team_tradex.xlsx    ← Source of truth for agents/TL/Pool
|
├─ stringeex_django/    ← Django project package
|   ├── settings.py       ← Reads .env, DB config, timezone, middleware
|   ├── urls.py           ← Root router: /admin/ and everything else →
calls.urls
|   └─ wsgi.py
|
└─ calls/               ← Main app
    ├── models.py         ← CallRecord model + display properties
    ├── views.py          ← All API endpoints + dashboard view
    ├── urls.py           ← URL patterns for all /api/* routes
    └─ team_lookup.py     ← Excel reader, in-memory cache, prefix matching

```

```

├── stringeex_service.py    ← HTTP client for StringeeX API (multi-portal)
├── auth_middleware.py     ← Token-based session auth
├── admin.py               ← Django admin registration
├── apps.py                ← App config
|
├── templates/calls/
|   ├── dashboard.html     ← 1460-line single-file dashboard (CSS + HTML +
JS)
|
├── migrations/
|   ├── 0001_initial.py    ← Creates CallRecord table
|   └── 0002_add_team_fields.py ← Adds full_name, employee_code, pool, tl,
vertical
|
└── management/commands/
    ├── sync_calls.py      ← Main ingestion command (API + CSV + daemon)
    └── reenrich_teams.py  ← Bulk re-apply Excel changes to existing DB
rows

```

3. Environment & Configuration

All config lives in `.env` at the project root (same directory as `manage.py`).

```
env
```

```
# Django core
DJANGO_SECRET_KEY=<long random string>
DEBUG=False
ALLOWED_HOSTS=localhost,127.0.0.1,192.168.155.204,.ngrok-free.dev

# PostgreSQL
DB_NAME=cdr
DB_USER=admin
DB_PASSWORD=Admin@nexora5
DB_HOST=localhost
DB_PORT=5432

# StringeeX API credentials
STRINGEEX_EMAIL=rahulnexora@gmail.com
STRINGEEX_PASSWORD=Abcd@1234

# Portals to sync from (comma-separated). Preferred over STRINGEEX_PORTAL.
STRINGEEX_PORTALS=bluewave2,bluewave3

# Single portal fallback (only used if STRINGEEX_PORTALS is blank)
STRINGEEX_PORTAL=bluewave2

# Daemon poll interval in minutes
POLL_INTERVAL_MINUTES=4

# Dashboard token auth (leave blank to disable)
ACCESS_TOKEN=M_1CWscVl09u0Y036o3yXyB7T0CoRoB1ssIaYvbKCfw
```

To add a new portal: append to `STRINGEEX_PORTALS`:

```
env

STRINGEEX_PORTALS=bluewave2,bluewave3,bluewave4
```

No code change needed — `stringeex_service.py` reads the list dynamically.

To change the team Excel path:

```
env

TEAM_XLSX_PATH=/absolute/path/to/Team_tradex.xlsx
```

Settings that are hardcoded (not in .env):

Setting	Value	File
TIME_ZONE	Asia/Kolkata	settings.py
SESSION_COOKIE_AGE	2 weeks	settings.py
Daemon overlap window	5 extra minutes	sync_calls.py:_run_daemon()

4. Database Model

Table name: `calls_callrecord`

python

```
class CallRecord(models.Model):
    id = CharField(primary_key=True) # StringeeX call ID e.g. "call-
    account_name = CharField() # Agent email e.g. "irfan@bluew
    direction = CharField() # 'inbound' | 'outbound' | 'int
    start_time_ms = BigIntegerField() # Unix timestamp in MILLISECOND
    end_time_ms = BigIntegerField() # Unix timestamp in MILLISECOND
    duration = IntegerField() # Total call seconds
    talk_duration = IntegerField() # Agent talk seconds (0 = not c
    answer_duration = FloatField() # Same as talk for connected ca
    end_call_reason = CharField() # 'USER_END_CALL', '404 Not Fou
    customer_number = CharField() # Customer phone number
    stringee_number = CharField() # Dialer's number
    # Team-enriched fields (written at ingest time, updatable via reenrich_teams)
    full_name = CharField() # From Excel
    employee_code = CharField() # From Excel
    pool = CharField() # From Excel e.g. 'Meta', 'Pull
    tl = CharField() # From Excel e.g. 'Rehan', 'Par
    vertical = CharField() # From Excel e.g. 'TradeX'
    fetched_at = DateTimeField(auto_now_add=True) # When WE stored it
```

Key computed properties (Python only, not DB columns):

python

```
record.start_datetime      # datetime object in UTC
record.queue_duration_secs # max(duration - talk_duration, 0)
record.is_connected        # talk_duration > 0
record.talk_duration_display # "HH:MM:SS"
record.total_duration_display # "HH:MM:SS"
record.queue_duration_display # "HH:MM:SS"
```

Critical facts:

- `id` is a STRING primary key, never an auto-increment integer. Django's `get_or_create(id=call_id)` handles deduplication.
- `start_time_ms` is milliseconds, not seconds. Divide by 1000 for Python `datetime.fromtimestamp()`.
- `talk_duration = 0` means the call was NOT answered (connect rate uses `talk_duration > 0`).
- `duration` is total call wall-clock time; `talk_duration` is agent-on-line time. Queue time = `duration - talk_duration`.

Useful raw SQL queries:

```
sql

-- Count today's calls (IST midnight = UTC 18:30 previous day)
SELECT COUNT(*) FROM calls_callrecord
WHERE start_time_ms >= 1746735000000
      AND start_time_ms <= 1746821399000;

-- Check if a specific agent has data
SELECT account_name, tl, pool, COUNT(*) as calls
FROM calls_callrecord
WHERE account_name ILIKE '%irfan%'
GROUP BY account_name, tl, pool;

-- Find all distinct end_call_reasons
SELECT end_call_reason, COUNT(*) FROM calls_callrecord GROUP BY end_call_reason ORDER BY end_call_reason;

-- Find records with missing team enrichment
SELECT account_name, COUNT(*) FROM calls_callrecord WHERE tl IS NULL GROUP BY account_name;
```

5. Team Lookup System

File: `calls/team_lookup.py`

Excel: `data/Team_tradex.xlsx`

How Matching Works

The API sends `account_name = "irfan@bluewave.com"`. The Excel has two relevant email types:

- Sheet1 `Email` column: `irfan@nexora.live` (official internal email)
- Sheet2 `Dialer Name` column: `irfan@bluewave.com` (the actual API email)

These are the same person with two different domains. **Prefix matching** solves this: both contain prefix `irfan`. The lookup is keyed on the lowercase prefix before `@`.

Build Order (Priority)

```
Sheet1 (Email column)    → loads all 124+ CRE agents by nexora.live prefix
    ↓ OVERRIDDEN BY
Sheet2 Stringee rows      → loads ~35 Stringee agents by bluewave.com prefix
                          (Sheet2 wins because it has the correct TL/Pool for
                          Stringee)
```

If an agent is in Sheet1 but NOT in Sheet2 (e.g. Tata dialer agents, OJT agents), Sheet1 is the source.

If an agent is in both, Sheet2 wins — and blank fields in Sheet2 are backfilled from Sheet1.

In-Memory Cache

```
python

_CACHE: dict | None = None # module-level singleton

# Populated on first call to get_team_lookup()
# Format: { 'irfan': {'full_name': 'Irfan', 'tl': 'Rehan', 'pool': 'Meta', ...} }
```

The cache is built **once at startup** and held in RAM. It is NOT invalidated automatically. To refresh without restart:

```
bash
```

```
# Hit the reload endpoint:
curl http://localhost:8000/api/reload-team/

# Or run the reenrich command (also reloads + rewrites DB):
python manage.py reenrich_teams
```

Excel Sheet Requirements

Sheet2 columns (Stringee rows):

Column	Description	Example
Dialer	Must be Stringee (or Strigee — typo handled)	Stringee
Dialer Name	API account_name or prefix	irfan@bluewave.com
Email	Official nexora.live email	irfan@nexora.live
Employee code	Staff ID	E-1234
Full Name	Display name	Irfan Khan
Pool	Team pool	Meta
TL	Team leader name	Rehan
Vertical	Business vertical	TradeX

Sheet1 columns:

Column	Description
Email	nexora.live email (key column)
Employee Code	Staff ID
Unnamed: 2 OR Full Name	Display name (code handles both)
Pool	Team pool
TL	Team leader
Vertical	Vertical

Typo tolerance: `Strigee` (missing 'n') is handled in the filter:

```
python
```

```
df2['Dialer'].str.lower().isin(['stringee', 'strigee'])
```

6. Data Ingestion — API & CSV

sync_calls Command Reference

```
bash
```

```
# ONE-SHOT: Sync last 2 hours from all portals (good for manual check)
```

```
python manage.py sync_calls
```

```
# ONE-SHOT with custom window
```

```
python manage.py sync_calls --hours 6
```

```
# DAEMON: Live polling every 4 min (production mode)
```

```
python manage.py sync_calls --daemon
```

```
python manage.py sync_calls --daemon --interval 4
```

```
# BACKFILL: Pull a date range (day-by-day to avoid timeouts)
```

```
python manage.py sync_calls --start 2026-05-01 --end 2026-05-09
```

```
# FLUSH then BACKFILL (nuclear option – wipes everything first)
```

```
python manage.py sync_calls --flush --start 2026-05-01 --end 2026-05-09
```

```
# FLUSH only (clear DB without syncing)
```

```
python manage.py sync_calls --flush
```

```
# CSV IMPORT
```

```
python manage.py sync_calls --csv /path/to/call_report_20260508_20260508.csv
```

Upsert Logic

```
python
```

```
def upsert_records(rows, replace_today=False):
    # SKIP: CAN_NOT_MAKE_CALL (filtered at write time)
    # Note: 404 Not Found is NOT filtered at ingest – only filtered in the dashboard

    if replace_today and _is_today_local(start_ms):
        # Live sync: update_or_create – keeps data fresh for today
        CallRecord.objects.update_or_create(id=call_id, defaults=fields)
    else:
        # Backfill / CSV: get_or_create – never overwrites historical data
        CallRecord.objects.get_or_create(id=call_id, defaults=fields)
```

Important filter difference:

- `CAN_NOT_MAKE_CALL` → filtered at **ingest** (never stored in DB)
- `404 Not Found` → stored in DB, filtered at **query time** in `api_hourly_breakdown()`

This means if you query the DB directly, you'll see `404 Not Found` records. The dashboard excludes them.

CSV Column Mapping

The CSV from the StringeeX export uses these column names:

CSV column	DB field	Notes
<code>id</code>	<code>id</code>	Primary key
<code>account_name</code>	<code>account_name</code>	Agent API email
<code>direction</code>	<code>direction</code>	Int <code>0</code> / <code>1</code> → mapped to <code>outbound</code> / <code>inbound</code>
<code>start_time</code>	<code>start_time_ms</code>	Unix ms (note: CSV field = <code>start_time</code> , DB field = <code>start_time_ms</code>)
<code>end_time</code>	<code>end_time_ms</code>	Unix ms
<code>duration</code>	<code>duration</code>	Seconds
<code>talk_duration</code>	<code>talk_duration</code>	Seconds
<code>answer_duration</code>	<code>answer_duration</code>	Float seconds, nullable
<code>end_call_reason</code>	<code>end_call_reason</code>	String
<code>customer_number</code>	<code>customer_number</code>	Phone
<code>stringee_number</code>	<code>stringee_number</code>	Dialer number

The CSV `start_time` field maps to `row.get('start_time')` — NOT `start_time_ms`.

This is the key difference from the API payload which uses `start_time` directly.

The `upsert_records()` function reads `row.get('start_time')` for both CSV and API rows.

Multi-Portal Fetching

`stringeex_service.py` authenticates separately for each portal and merges results. If one portal fails, the others continue. Records from different portals can have the same `id` — deduplication happens at the `get_or_create`/`update_or_create` call.

```
python
```

```
# Authentication: POST /v1/account
# Portal matching: response.portals[].domain == portal_name (plain string, not full
# Call history: GET /v1/call/history?limit=1000&page=N&start_time=ms&end_time=ms
```

7. API Endpoints Reference

All endpoints are `GET` only. All return `application/json`.

`GET /api/hourly-breakdown/`

The main dashboard endpoint. Powers the hourly table.

Parameters:

Param	Type	Description
<code>date</code>	<code>YYYY-MM-DD</code>	Target date (default: today in IST)
<code>tl</code>	string	Filter by TL name (case-insensitive)
<code>pool</code>	string	Filter by Pool name (case-insensitive)
<code>agent</code>	string	Filter to single agent email (case-insensitive)

Filters applied server-side:

- Excludes `end_call_reason IN ('404 Not Found', 'CAN_NOT_MAKE_CALL')`
- Date range computed as IST midnight → IST 23:59:59, then converted to UTC ms for the DB query

Response shape:

json

```

{
  "hours": [9, 10, 11, 12],
  "date": "2026-05-09",
  "rows": [
    {
      "account_name": "irfan@bluewave.com",
      "full_name": "Irfan",
      "employee_code": "E-1234",
      "pool": "Meta",
      "tl": "Rehan",
      "vertical": "TradeX",
      "hourly": {
        "9": {"calls": 12, "unique_dials": 10, "duration": 480, "talk_duration": 240},
        "10": {"calls": 8, "unique_dials": 7, "duration": 360, "talk_duration": 180},
      },
      "totals": {"calls": 20, "unique_dials": 17, "duration": 840, "talk_duration": 420},
    }
  ],
  "hour_totals": {
    "9": {"calls": 100, "unique_dials": 85, "duration": 5000, "talk_duration": 2400},
    "10": {"calls": 80, "unique_dials": 70, "duration": 4000, "talk_duration": 1800},
  }
}

```

Note: `hours` are integers (9, not "09"). The JS formats them with `String(h).padStart(2, '0') + ':00'`.

GET /api/team-list/

Powers the sidebar and filter dropdowns. Called once on page load.

Response:

json

```

{
  "tls": ["Nirvik", "Parth", "Rehan", "Roshni", "Saif", "Saloni", "Swathi", "Vikas s",
  "pools": ["Customer care", "Dialer", "Meta", "OJT", "Pull Back", "Welcome Calling"],
  "agents": [
    {"account_name": "irfan@bluewave.com", "full_name": "Irfan", "tl": "Rehan", "poo
  ]
}

```

Data sources: merged from Excel (all agents) + DB (actual API emails). Excel agents show their prefix as `account_name` until a DB record updates it to the real email.

`GET /api/stats/`

KPI aggregates (total, connected, connect rate, talk time).

Parameters: `date`, `days`, `agent`, `tl`

`GET /api/calls/`

Paginated raw call log.

Parameters: `date`, `limit` (max 500), `offset`, `direction`, `agent`, `tl`, `connected=1`, `search`

`GET /api/agent-summary/`

Per-agent totals (no hourly breakdown).

Parameters: `date`, `days`, `tl`

`GET /api/last-sync/`

Returns `{"last_sync": "2026-05-09T09:42:17.123456+00:00"}` — the `fetches_at` timestamp of the most recently stored record.

`GET /api/reload-team/`

Force-reloads `Team_tradex.xlsx` into memory. Returns `{"status": "ok", "agents_loaded": 124}`. Use this after updating the Excel without restarting the server.

8. Dashboard Frontend

File: `calls/templates/calls/dashboard.html` (1460 lines, self-contained)

State Object

```
javascript
```

```
const state = {
  view: 'table',          // 'table' | 'cards'
  activeAgent: null,      // null = all agents, or email string
  tl: '', pool: '', date: '',
  metrics: { calls: true, unique: true, talk: true, dur: true },
  breakdownData: null,    // last /api/hourly-breakdown/ response
  allAgents: [],          // from /api/team-list/
  countdown: 240,         // seconds until next auto-refresh
  countdownTimer: null,
};
```

Refresh Cycle

```
DOMContentLoaded → loadTeamList() + loadBreakdown()
                    ↓
                    startCountdown()
                    ↓ (every second)
                    countdown -= 1
                    ↓ (when 0)
                    refreshNow() → loadBreakdown() + loadLastSync() +
                    startCountdown()
```

Auto-refresh only re-fetches `hourly-breakdown` and `last-sync`. The team list (`team-list`) is fetched once on load only.

Key JS Functions

Function	Purpose
<code>loadBreakdown()</code>	Fetches <code>/api/hourly-breakdown/</code> , calls <code>renderKPIs</code> + <code>renderBreakdown</code> + <code>renderSummaryCards</code>
<code>renderBreakdown(data)</code>	Builds the hourly table HTML from scratch (no library)
<code>renderAgentList(agents)</code>	Re-renders the sidebar agent list (called after each breakdown load to show call counts)
<code>selectAgent(email)</code>	Sets <code>state.activeAgent</code> , re-runs <code>loadBreakdown()</code> , shows call log
<code>loadCallLog()</code>	Fetches <code>/api/calls/</code> with current filters, appends to tbody
<code>exportCallLogExcel()</code>	Fetches ALL matching calls (no limit), uses SheetJS to build .xlsx

Function	Purpose
<code>exportBreakdownExcel()</code>	Serialises current <code>state.breakdownData</code> to .xlsx using SheetJS
<code>applyFilters()</code>	Reads DOM filter values → updates state → calls <code>loadBreakdown()</code>
<code>toggleMetric(key)</code>	Toggles a column group in the hourly table, calls <code>renderBreakdown()</code>
<code>maskNumber(num)</code>	Privacy masking: shows last 4 digits only e.g. <code>+91XXXXXX1234</code>

Data Format Notes for JS

- `hours` from the API are integers. Use `String(h).padStart(2, '0') + ':00'` to display.
- Durations from the API are always **seconds** (integers). Use `secsToHMS(secs)` to convert.
- `hourly` is a plain object keyed by integer hour as a string (`"9"`, `"10"`). Always access as `r.hourly[h]` where `h` is the integer.
- `hour_totals` is similarly keyed.

Adding a New Column to the Hourly Table

- Fetch the extra field in `api_hourly_breakdown()` (add to the `records.values(...)` call in `views.py`)
- Aggregate it in the `for r in records:` loop, add to the bucket dict
- Include it in `hourly[h]` and `agent_total`
- In `renderBreakdown()` in the JS, add a new `.m-X` span inside the `cell-metric` div
- Add a toggle chip in the HTML `col-toggle` section and handle it in `toggleMetric()`

9. Access Control

File: `calls/auth_middleware.py`

Token auth is session-based. The flow:

- Admin shares `https://yourdomain.com/?token=<ACCESS_TOKEN>` with users.
- User visits the URL. Middleware reads `?token=...`, validates against `.env`'s `ACCESS_TOKEN`.
- If valid → sets `request.session['authenticated'] = True` (14-day session).
- On all subsequent requests → only checks the session cookie. Token disappears from URL.

Bypass rules:

- `/admin/*` → always allowed (uses Django's own auth)
- `/api/*` → allowed only if session is authenticated (returns 403 otherwise)
- Everything else → requires session or valid token query param

To disable auth (dev/internal only): set `ACCESS_TOKEN=` (blank) in `.env`.

To rotate the token: update `ACCESS_TOKEN` in `.env`, restart Django. Old sessions remain valid until they expire (14 days). To force everyone to re-authenticate, also clear the session table:

```
bash  
  
python manage.py clearsessions
```

10. Timezone Handling

This is the most common source of bugs. Read carefully.

All timestamps in the DB are stored as **UTC milliseconds** (`start_time_ms`, `end_time_ms`).

All display and date boundaries use **IST (Asia/Kolkata, UTC+5:30)**.

The constant `LOCAL_TZ = ZoneInfo('Asia/Kolkata')` is defined in both `views.py` and `sync_calls.py`.

Date Boundary Calculation

When the user selects date `2026-05-09`, the query must cover IST midnight → IST 23:59:59, converted to UTC ms:

```
python  
  
# IST midnight 2026-05-09 = UTC 2026-05-08 18:30:00  
# IST 23:59:59 2026-05-09 = UTC 2026-05-09 18:29:59  
  
def _day_ms_range(date_str):  
    td = datetime.strptime(date_str, '%Y-%m-%d')  
    day_start = datetime(td.year, td.month, td.day, 0, 0, 0, tzinfo=LOCAL_TZ)  
    day_end    = datetime(td.year, td.month, td.day, 23, 59, 59, tzinfo=LOCAL_TZ)  
    return int(day_start.timestamp() * 1000), int(day_end.timestamp() * 1000)
```

python

```
def _ms_to_local_hour(ms):  
    return datetime.fromtimestamp(ms / 1000, tz=LOCAL_TZ).hour
```

A call at UTC 03:30 is at IST 09:00 → bucket 9.

Never bucket by UTC hour — a call at UTC 03:30 is NOT a 3am call; it's a 9am IST call.

Frontend Date Picker

The date picker defaults to `new Date().toISOString().split('T')[0]` which is UTC date. If the user is in IST and it's past midnight IST but before midnight UTC, this will show yesterday's date. For accuracy, compute local date properly:

javascript

```
// Current code (may show UTC date):  
const today = new Date().toISOString().split('T')[0];  
  
// IST-correct version if needed in future:  
const now = new Date();  
const ist = new Date(now.getTime() + (5.5 * 60 * 60 * 1000));  
const today = ist.toISOString().split('T')[0];
```

11. Debugging Runbook

11.1 Dashboard Shows No Data

Step 1: Check the DB has records for the date.

bash

```
# Django shell  
python manage.py shell  
from calls.models import CallRecord  
from django.db.models import Count  
CallRecord.objects.all().count() # total records  
  
# Check for today's records (replace ms values with actual IST midnight UTC)  
CallRecord.objects.filter(start_time_ms__gte=1746735000000).count()
```

Step 2: Check the API endpoint directly.

bash

```
curl "http://localhost:8000/api/hourly-breakdown/?date=2026-05-09"
# Expected: {"hours":[...], "rows":[...]}
# If 403: your session is not authenticated
# If 500: check Django logs
```

Step 3: Check the browser console.

- Open DevTools → Network → look for `/api/hourly-breakdown/` call
- Status 200 with empty `rows: []` → data not in DB for that date
- Status 403 → auth issue
- Status 500 → server error (check logs)

Step 4: Check if all rows are filtered out.

python

```
# Are ALL records for today excluded by the filter?
from calls.models import CallRecord
qs = CallRecord.objects.filter(start_time_ms__gte=1746735000000)
print(qs.values('end_call_reason').annotate(n=Count('id')))
# If only '404 Not Found' and 'CAN_NOT_MAKE_CALL' exist, dashboard will be blank
```

11.2 Agent Shows Wrong TL / Pool

Step 1: Check what the DB has for that agent.

python

```
from calls.models import CallRecord
qs = CallRecord.objects.filter(account_name__iexact='irfan@bluewave.com')
print(qs.values('account_name', 'full_name', 'tl', 'pool').distinct())
```

Step 2: Check what the team lookup resolves to.

python

```
from calls.team_lookup import enrich_record, reload_team_lookup
reload_team_lookup() # force fresh read from Excel
info = enrich_record('irfan@bluewave.com')
print(info) # {'full_name': 'Irfan', 'tl': 'Rehan', 'pool': 'Meta', ...}
```

Step 3: Check the lookup cache directly.

```
python

from calls.team_lookup import get_team_lookup
lookup = get_team_lookup()
print(lookup.get('irfan')) # 'irfan' is the prefix
```

Step 4: If lookup is wrong, fix Excel and re-enrich.

```
bash

# Fix Team_tradex.xlsx, then:
python manage.py reenrich_teams --dry-run # preview changes
python manage.py reenrich_teams          # apply changes
```

11.3 Agent Not Appearing in Sidebar

The sidebar merges Excel agents with DB agents. An agent will appear if:

- They are in the Excel (Sheet1 or Sheet2), OR
- They have at least one `CallRecord` in the DB

Debugging:

```
python

# 1. Check if prefix is in lookup
from calls.team_lookup import get_team_lookup
lookup = get_team_lookup()
prefix = 'anushka' # the part before @
print(prefix in lookup) # True/False

# 2. Check DB
from calls.models import CallRecord
CallRecord.objects.filter(account_name__icontains='anushka').values('account_name').
```

If the agent made calls today but isn't in sidebar:

- Their `account_name` is in the DB
 - The prefix extraction in `api_team_list()` should pick them up
 - Check if their prefix collides with another agent
-

11.4 Hourly Timestamps Are Wrong (Off by Hours)

Symptom: Calls that happened at 9am show up under the 3am column.

Cause: UTC hour is being used instead of IST hour.

Verify: Check `_ms_to_local_hour()` in `views.py`. It must use `tz=LOCAL_TZ` (Asia/Kolkata):

```
python

return datetime.fromtimestamp(ms / 1000, tz=LOCAL_TZ).hour # CORRECT
return datetime.fromtimestamp(ms / 1000).hour             # WRONG - uses system tz
```

Check server timezone:

```
bash

timedatectl # or
date
```

If the server is in UTC (common for cloud VMs), naively calling `datetime.fromtimestamp()` without a `tz` argument gives UTC. Always pass `tz=LOCAL_TZ`.

11.5 Daemon Not Syncing / Missing Records

Check if daemon is running:

```
bash

ps aux | grep sync_calls
# or if using systemd:
sudo systemctl status stringeex-sync
sudo journalctl -u stringeex-sync -n 50 --no-pager
```

Check API connectivity:

```
bash
```

```
python manage.py sync_calls --hours 1 # one-shot test
```

Check for API auth errors:

```
python
```

```
from calls.stringeex_service import get_auth_token_for_portal
token, base_url = get_auth_token_for_portal('bluewave2')
print(token[:20]) # Should print part of the token
```

Portal matching issue: The API returns `domain = 'bluewave2'` (plain name). The code matches on plain name. If a new portal uses a different naming convention:

```
python
```

```
# In stringeex_service.py, get_auth_token_for_portal():
for p in portals:
    domain = str(p.get('domain', '')).strip()
    if domain == portal: # portal = 'bluewave2', domain = 'bluewave2' → matches
        token = p.get('auth_token')
```

Add a `print(portals)` to debug what the API actually returns.

Gap in data: The daemon fetches `interval + 5` minutes back on each cycle:

```
python
```

```
start = now - timedelta(minutes=interval + 5) # 9 minutes back for 4-min interval
```

If the daemon restarted and was down for > 9 minutes, there will be a gap. Fix with:

```
bash
```

```
python manage.py sync_calls --start 2026-05-09 --end 2026-05-09 # backfill the gap
```

11.6 `404 Not Found` Still Appearing in Dashboard

`404 Not Found` records are stored in the DB but excluded by the dashboard API. If you see them in the dashboard, check:

```
python
```

```
# views.py api_hourly_breakdown():
qs = CallRecord.objects.filter(...).exclude(
    end_call_reason__in=['404 Not Found', 'CAN_NOT_MAKE_CALL']
)
```

The string must match exactly (case-sensitive, with space). Verify in DB:

```
python

CallRecord.objects.filter(end_call_reason='404 Not Found').count()
```

11.7 Excel Not Loading (Team Lookup Returns Empty)

```
python

from calls.team_lookup import _build_lookup, TEAM_XLSX_PATH
print('Path:', TEAM_XLSX_PATH) # check it's correct
lookup = _build_lookup(TEAM_XLSX_PATH)
print('Keys:', len(lookup))
```

Common causes:

- Excel file is open in another application → file lock → pandas fails silently
- Sheet name changed (must be `Sheet1` and `Sheet2` exactly)
- `Dialer` column has values other than `Stringee` / `Strigee` — check for trailing spaces
- `TEAM_XLSX_PATH` env var pointing to wrong path

```
python

import pandas as pd
xf = pd.ExcelFile('/path/to/Team_tradex.xlsx')
print(xf.sheet_names) # must include 'Sheet1' and 'Sheet2'
df2 = xf.parse('Sheet2')
print(df2['Dialer'].unique()) # should include 'Stringee'
```

11.8 `CAN_NOT_MAKE_CALL` Records Appearing in DB

If you find `CAN_NOT_MAKE_CALL` in the DB, they were inserted before the filter was added, or via a raw SQL insert. Clean up:

```
python
```

```
CallRecord.objects.filter(end_call_reason='CAN_NOT_MAKE_CALL').delete()
```

11.9 Database Migration Errors

```
bash
```

```
python manage.py migrate --check # check if migrations are pending
```

```
python manage.py showmigrations # see which are applied
```

```
# Fake a migration if it was applied manually:
```

```
python manage.py migrate calls 0002 --fake
```

If `answer_duration` gives type errors: migration `0002` changes it from `IntegerField` to `FloatField`. If the migration wasn't applied, do so:

```
bash
```

```
python manage.py migrate calls 0002
```

11.10 Django Admin Debug

Access at `http://localhost:8000/admin/` with your Django superuser.

```
bash
```

```
python manage.py createsuperuser
```

From admin you can:

- Browse all `CallRecord` rows with search and filter
- Search by `customer_number`, `account_name`, `stringee_number`
- Filter by `direction`, `end_call_reason`
- The `duration_display` computed property is shown in the list

11.11 Checking Logs

```
bash
```

```
# If running via systemd:
sudo journalctl -u stringeex-web -n 100 --no-pager
sudo journalctl -u stringeex-sync -n 100 --no-pager

# Django dev server logs are in the terminal where you ran manage.py runserver
# For gunicorn, check your access/error logs
```

Enable Django SQL logging (dev only — very verbose):

```
python

# In settings.py, add:
LOGGING = {
    'version': 1,
    'handlers': {'console': {'class': 'logging.StreamHandler'}},
    'loggers': {
        'django.db.backends': {'handlers': ['console'], 'level': 'DEBUG'},
        'calls': {'handlers': ['console'], 'level': 'DEBUG'},
    }
}
```

12. Team List Updates

Scenario A: Add a New Agent

1. Open `data/Team_tradex.xlsx`
2. **Sheet2:** Add a new row with `Dialer = Stringee`, fill all columns
3. Save and close Excel
4. Reload the lookup (no restart needed):

```
bash

curl http://localhost:8000/api/reload-team/
# Response: {"status": "ok", "agents_loaded": 125}
```

5. New agent will appear in sidebar on next page load
6. Re-enrich existing DB records if the agent has historical data:

```
bash
```

```
python manage.py reenrich_teams
```

Scenario B: Change a TL Assignment

1. Edit `data/Team_tradex.xlsx` — update `TL` column for the agent(s) in Sheet2
2. Save and close
3. Reload lookup + re-enrich DB:

```
bash
```

```
curl http://localhost:8000/api/reload-team/  
python manage.py reenrich_teams
```

4. Verify:

```
python
```

```
from calls.models import CallRecord  
CallRecord.objects.filter(account_name__iexact='irfan@bluewave.com').values('tl')
```

Scenario C: Add a New Pool

No code change needed. Just add the pool name to the relevant rows in Excel. The `pool`s list in `/api/team-list/` is built dynamically from the data.

Scenario D: Rename a TL

1. Update all rows in Excel with the old TL name to the new name
2. Reload and re-enrich:

```
bash
```

```
curl http://localhost:8000/api/reload-team/  
python manage.py reenrich_teams
```

3. The TL filter dropdown will show the new name on next page load

Scenario E: Agent Email Changes (e.g. from bluewave.com to nexora.live)

The prefix matching handles this automatically as long as the prefix before `@` stays the same.

If the prefix also changes (e.g. `anushka` becomes `anushkak`):

1. Update Sheet2 `Dialer Name` to new email
2. Add the old prefix as an alias by adding a second Sheet2 row with the old `Dialer Name`
3. Reload and re-enrich

`reenrich_teams` — What It Does

bash

```
python manage.py reenrich_teams --dry-run # Preview: how many rows would change
python manage.py reenrich_teams          # Apply changes
```

- Reloads Excel into cache first
- Iterates all `CallRecord` rows in batches of 500
- Calls `enrich_record(account_name)` for each
- Only updates rows where TL/Pool/Full Name has changed (efficient)
- Uses `bulk_update` for performance

Runtime: ~2-3 seconds per 10,000 records on a normal PostgreSQL instance.

13. Adding New Panels & Endpoints

Adding a New API Endpoint

Example: Top 5 agents by connected calls today

Step 1: Add view function in `calls/views.py`:

python

```

@require_GET
def api_top_agents(request):
    date_str = request.GET.get('date', '').strip()
    tl       = request.GET.get('tl', '').strip()

    qs = CallRecord.objects.all()
    if date_str:
        s, e = _day_ms_range(date_str)
        qs = qs.filter(start_time_ms__gte=s, start_time_ms__lte=e)
    if tl:
        qs = qs.filter(tl__iexact=tl)

    qs = qs.exclude(end_call_reason__in=['404 Not Found', 'CAN_NOT_MAKE_CALL'])

    top = (
        qs.values('account_name', 'full_name', 'tl', 'pool')
        .annotate(
            total_calls=Count('id'),
            connected=Count('id', filter=Q(talk_duration__gt=0)),
            total_talk=Sum('talk_duration'),
        )
        .order_by('-connected')[:5]
    )

    rows = [{
        'account_name': a['account_name'],
        'full_name':    a['full_name'] or a['account_name'],
        'tl':           a['tl'] or '-',
        'pool':         a['pool'] or '-',
        'total_calls':  a['total_calls'],
        'connected':    a['connected'],
        'connect_rate': round(a['connected'] / a['total_calls'] * 100, 1) if a['total_calls'] > 0 else 0,
        'talk_time':    _secs_to_hhmmss(a['total_talk'] or 0),
    } for a in top]

    return JsonResponse({'rows': rows})

```

Step 2: Register URL in `calls/urls.py`:

python

```
path('api/top-agents/', views.api_top_agents, name='api_top_agents'),
```

Step 3: Call from JS in `dashboard.html`:

javascript

```
async function loadTopAgents() {
  const params = new URLSearchParams();
  if (state.date) params.set('date', state.date);
  if (state.tl) params.set('tl', state.tl);
  const res = await fetch(`/api/top-agents/?${params}`);
  const data = await res.json();
  renderTopAgents(data.rows);
}

function renderTopAgents(rows) {
  const wrap = document.getElementById('top-agents-panel');
  wrap.innerHTML = rows.map(r => `
    <div class="agent-rank-card">
      <span class="rank-name">${escH(r.full_name)}</span>
      <span class="rank-connected">${r.connected} connected</span>
      <span class="rank-rate">${r.connect_rate}%</span>
    </div>
  `).join('');
}
```

Step 4: Add the HTML container in `dashboard.html`:

html

```
<div id="top-agents-panel" class="panel">
  <!-- filled by renderTopAgents() -->
</div>
```

Step 5: Call `loadTopAgents()` inside `loadBreakdown()` so it refreshes with the rest:

javascript

```
async function loadBreakdown() {
  // ... existing code ...
  loadTopAgents(); // add this line
}
```

Adding a New Model Field

1. Add the field to `models.py`:

python

```
new_field = models.CharField(max_length=64, blank=True, null=True)
```

2. Create a migration:

bash

```
python manage.py makemigrations calls -n 'add_new_field'  
python manage.py migrate
```

3. Add it to `upsert_records()` `defaults` dict in `sync_calls.py`

4. Add it to the `fields` dict in `upsert_records()`:

python

```
'new_field': row.get('new_field_api_key'),
```

5. Expose in `api_calls()` response dict if needed

6. Add to `reenrich_teams.py` if it's team metadata

Adding a New Chart

The dashboard uses no charting library for the hourly table (pure DOM). For a new chart, SheetJS is already loaded. For a chart, add Chart.js or use a simple canvas approach.

Pattern for a bar chart panel:

html

```
<canvas id="hourly-chart" width="600" height="200"></canvas>
<script src="https://cdnjs.cloudflare.com/ajax/libs/Chart.js/4.4.0/chart.umd.min.js"
<script>
let hourlyChart = null;

function renderHourlyChart(data) {
  const hours = data.hours.map(h => String(h).padStart(2, '0') + ':00');
  const calls  = data.hours.map(h => data.hour_totals[h]?.calls || 0);

  if (hourlyChart) hourlyChart.destroy();
  hourlyChart = new Chart(document.getElementById('hourly-chart'), {
    type: 'bar',
    data: {
      labels: hours,
      datasets: [{ label: 'Calls', data: calls, backgroundColor: '#64ffda' }]
    }
  });
}
</script>
```

Then call `renderHourlyChart(data)` inside `loadBreakdown()`.

14. Data Change Operations

Wipe and Rebuild for a Date Range

```
bash

# Step 1: Wipe all data
python manage.py sync_calls --flush

# Step 2: Backfill from API
python manage.py sync_calls --start 2026-05-01 --end 2026-05-09

# Step 3: Re-enrich team data
python manage.py reenrich_teams
```

Delete Only Today's Records (to Re-sync)

```
python
```

```
# Django shell
from calls.models import CallRecord
from datetime import datetime
from zoneinfo import ZoneInfo

LOCAL_TZ = ZoneInfo('Asia/Kolkata')
today = datetime.now(tz=LOCAL_TZ)
day_start = datetime(today.year, today.month, today.day, 0, 0, 0, tzinfo=LOCAL_TZ)
day_end = datetime(today.year, today.month, today.day, 23, 59, 59, tzinfo=LOCAL_TZ)

start_ms = int(day_start.timestamp() * 1000)
end_ms = int(day_end.timestamp() * 1000)

count, _ = CallRecord.objects.filter(
    start_time_ms__gte=start_ms,
    start_time_ms__lte=end_ms
).delete()
print(f'Deleted {count} records')
```

Import a Specific CSV File

```
bash
```

```
python manage.py sync_calls --csv /home/admin/call_report_20260509_20260509.csv
```

This uses `get_or_create` — it will not overwrite existing records with the same `id`.

Fix Missing Team Data on Specific Records

```
python
```

```
# Shell: fix TL/Pool for one agent's records
from calls.models import CallRecord
from calls.team_lookup import reload_team_lookup, enrich_record

reload_team_lookup()
info = enrich_record('newagent@bluewave.com')
updated = CallRecord.objects.filter(account_name__iexact='newagent@bluewave.com').update(
    full_name      = info['full_name']      or None,
    employee_code  = info['employee_code']  or None,
    pool           = info['pool']           or None,
    tl             = info['tl']             or None,
    vertical       = info['vertical']       or None,
)
print(f'Updated {updated} records')
```

Add a Custom end_call_reason Filter

Currently the dashboard excludes `404 Not Found` and `CAN_NOT_MAKE_CALL`. To add more:

In `views.py`, `api_hourly_breakdown()`:

```
python

EXCLUDED_REASONS = ['404 Not Found', 'CAN_NOT_MAKE_CALL', '503 Service Unavailable']

qs = CallRecord.objects.filter(...).exclude(
    end_call_reason__in=EXCLUDED_REASONS
)
```

15. Deployment & Process Management

Typical Process Setup (systemd)

Two services should run:

Web server (`/etc/systemd/system/stringeex-web.service`):

```
ini
```

```
[Unit]
Description=StringeeX Django Web
After=postgresql.service

[Service]
User=appuser
WorkingDirectory=/opt/stringeex_django
ExecStart=/opt/stringeex_env/bin/gunicorn stringeex_django.wsgi:application \
    --bind 0.0.0.0:8000 --workers 2 --timeout 60
Restart=always
EnvironmentFile=/opt/stringeex_django/.env

[Install]
WantedBy=multi-user.target
```

Sync daemon (`/etc/systemd/system/stringeex-sync.service`):

```
ini

[Unit]
Description=StringeeX Sync Daemon
After=postgresql.service stringeex-web.service

[Service]
User=appuser
WorkingDirectory=/opt/stringeex_django
ExecStart=/opt/stringeex_env/bin/python manage.py sync_calls --daemon --interval 4
Restart=on-failure
RestartSec=30
EnvironmentFile=/opt/stringeex_django/.env

[Install]
WantedBy=multi-user.target
```

```
bash

sudo systemctl enable stringeex-web stringeex-sync
sudo systemctl start stringeex-web stringeex-sync
sudo systemctl status stringeex-web stringeex-sync
```

Initial Setup Sequence

```
bash

cd /opt/stringeex_django
python -m venv /opt/stringeex_env
source /opt/stringeex_env/bin/activate
pip install -r requirements.txt

# Configure .env (copy and edit)
cp .env.example .env
nano .env

# Run migrations
python manage.py migrate

# Create admin user
python manage.py createsuperuser

# Collect static files (for production)
python manage.py collectstatic --no-input

# Test one-shot sync
python manage.py sync_calls --hours 4

# Start services
sudo systemctl start stringeex-web stringeex-sync
```

After Deploying Code Changes

```
bash

sudo systemctl restart stringeex-web

# If model changes were made:
python manage.py migrate
sudo systemctl restart stringeex-web

# If Excel was updated:
curl http://localhost:8000/api/reload-team/
python manage.py reenrich_teams
```

Monitoring

```
bash

# Live sync daemon log
sudo journalctl -u stringeex-sync -f

# Check DB size
psql -U admin -d cdr -c "SELECT COUNT(*) FROM calls_callrecord;"

# Check recent fetches
psql -U admin -d cdr -c "SELECT fetched_at, COUNT(*) FROM calls_callrecord GROUP BY
```

16. Common Gotchas & Known Quirks

1. `start_time` vs `start_time_ms`

The CSV column is named `start_time`. The DB field is `start_time_ms`. In `upsert_records()`, the code reads `row.get('start_time')` (NOT `row.get('start_time_ms')`). This is intentional — the raw dict from the API and from `df.to_dict()` both have `start_time`. The model stores it as `start_time_ms`. Don't rename either.

2. `CAN_NOT_MAKE_CALL` vs `404 Not Found` — Different Filter Levels

Reason	Where filtered
<code>CAN_NOT_MAKE_CALL</code>	At ingest → never stored in DB
<code>404 Not Found</code>	At query time → stored in DB, excluded by API

Consequence: `CallRecord.objects.all().count()` will include `404 Not Found` records but NOT `CAN_NOT_MAKE_CALL` records.

3. `answer_duration` vs `talk_duration`

Both represent the time the agent was connected. `answer_duration` comes directly from the API as a float and is sometimes `NaN` or `null`. `talk_duration` is always an integer and is the reliable field for "was this call connected?" checks. Always use `talk_duration > 0` for connection detection.

4. The Daemon Uses `replace_today=True`, Backfill Uses `False`

Live sync uses `update_or_create` for today's records (because call durations may not be finalised mid-call). Backfill uses `get_or_create` (never overwrite settled history). If you want to refresh historical data, use `--flush` first.

5. `api_team_list()` Returns Prefixes, Not Full Emails, for Excel-Only Agents

Until an agent makes a real call (which puts their actual email in the DB), their `account_name` in the team list response is their prefix (e.g., `irfan` instead of `irfan@bluewave.com`). The `selectAgent()` JS function passes this as the `?agent=` filter. If the prefix is `irfan` and the DB has `irfan@bluewave.com`, the `iexact` filter on `account_name` won't match.

The `api_team_list()` view resolves this by updating `by_prefix[prefix]['account_name']` to the real email when a DB record exists:

```
python

if prefix in by_prefix:
    by_prefix[prefix]['account_name'] = acc # use real API email
```

6. IST = Asia/Kolkata, NOT Asia/Dubai

Despite the project being for a Dubai-based operation, the code uses `Asia/Kolkata` (UTC+5:30). Dubai is UTC+4. This is a deliberate configuration choice (likely because the call centre operates on IST). Don't change this unless explicitly requested — it will shift all date boundaries and hourly labels.

7. Django `TIME_ZONE` Setting Doesn't Affect `start_time_ms`

`settings.py` has `TIME_ZONE = 'Asia/Kolkata'`. This affects Django's `DateTimeField` auto-rendering, but `start_time_ms` is a `BigIntegerField` (raw UTC milliseconds). Django doesn't apply any timezone conversion to it. All timezone logic is manual, in `views.py` and `sync_calls.py`.

8. Excel Cache Is Process-Scoped

`_CACHE` is a module-level variable. If Django runs with multiple workers (e.g., 2 gunicorn workers), each worker has its own `_CACHE`. Calling `/api/reload-team/` hits one worker — the other still has the old cache. Either restart all workers, or hit reload multiple times (crude), or use a shared cache like Redis in the future.

9. Call Log Pagination

The call log in the dashboard uses offset-based pagination. `callLogPage` is a module-level JS variable (not in `state`). If filters change while the log is open, reset with `callLogPage = 0;`

`loadCallLog()`. The "Load More" button increments `callLogPage`.

10. SheetJS Export Uses Current Filter State

Both export buttons (breakdown and call log) re-fetch the API with the current `state` filters. The breakdown export serialises `state.breakdownData` directly (no API call). The call log export fetches `/api/calls/?limit=10000` — a large limit to get everything. If there are more than 10,000 calls in the filter, the export will be incomplete. Increase the limit or use server-side export for large datasets.

11. Direction Field Is an Integer in the API, String in the DB

The StringeeX API returns `direction: 0` (outbound) or `direction: 1` (inbound). `upsert_records()` maps this to `'outbound'`/`'inbound'` before storing. If you import old data that already has string directions (`'outbound'`, `'inbound'`), the `_direction_from_int()` function handles that too via the `isinstance(val, str)` branch.

12. `fetch_at` Is When Django Stored It, Not When the Call Happened

`fetch_at` is the timestamp of when `CallRecord` was inserted/updated, not the call's actual time. Use `start_time_ms` for call timing. `fetch_at` is used in `/api/last-sync/` to show when the dashboard was last updated.

End of Developer Reference